UNIVERSITÀ
DELLA CALABRIA

DIPARTIMENTO DI INGEGNERIA INFORMATICA, MODELLISTICA, ELETTRONICA E SISTEMISTICA

Corso di Laurea in TELECOMMUNICATION ENGINEERING: SMART SENSING, COMPUTING AND NETWORKING

Enhancing Home Energy Efficiency with Predictive Appliance Control

Professor:
Floriano De Rango

Students:
Ali Allouf
Aurelio Benvenuto
Rama Mhalla

ACADEMIC YEAR 25/26

Contents

1 Introduction

The rapid proliferation of the Internet of Things (IoT) has fundamentally transformed the concept of the residential environment, evolving it from a static infrastructure into an interactive, data-driven ecosystem. However, contemporary smart home solutions often suffer from two major limitations: the fragmentation of communication protocols and the lack of proactive, intelligent energy management. Systems typically react to user commands or simple thresholds, failing to anticipate complex, non-linear energy demands based on human behavior and environmental shifts.

This project proposes the design, development, and implementation of an advanced Smart Home infrastructure that bridges the gap between heterogeneous IoT networking and Artificial Intelligence (AI). The core objective is to engineer a centralized Edge-Computing Gateway (powered by a Raspberry Pi) capable of orchestrating diverse wireless protocols, namely Zigbee for indoor mesh communication, LoRa for long-range outdoor telemetry, and BLE for secure localized access control.

Beyond simple remote actuation, the true innovation of this architecture lies in its predictive capabilities. By integrating a deep learning framework, specifically a Bidirectional Long Short-Term Memory (Bi-LSTM) neural network, the system acts as a proactive "Smart Meter". It continuously analyzes high-resolution environmental and appliance telemetry to forecast household energy loads. This enables the implementation of automated safety mechanisms, such as predictive load shedding, to prevent grid overloads before they physically occur.

To ensure a seamless user experience, the entire backend complexity is abstracted through a custom-built mobile application. Developed in Flutter and synchronized via MQTT, the dashboard provides real-time visualization, remote control, and AI-driven alerts, offering a holistic and secure interface for smart home management.

This report details the architectural choices, hardware implementation, predictive modeling, and software engineering required to build this integrated cyber-physical system.

2 Physical Architecture of the System (Smart Home)

The following diagram illustrates the complete physical deployment of the smart home ecosystem. It highlights the central role of the Raspberry Pi 5 Gateway and the distributed nature of the Arduino edge nodes across the different functional areas of the residence.

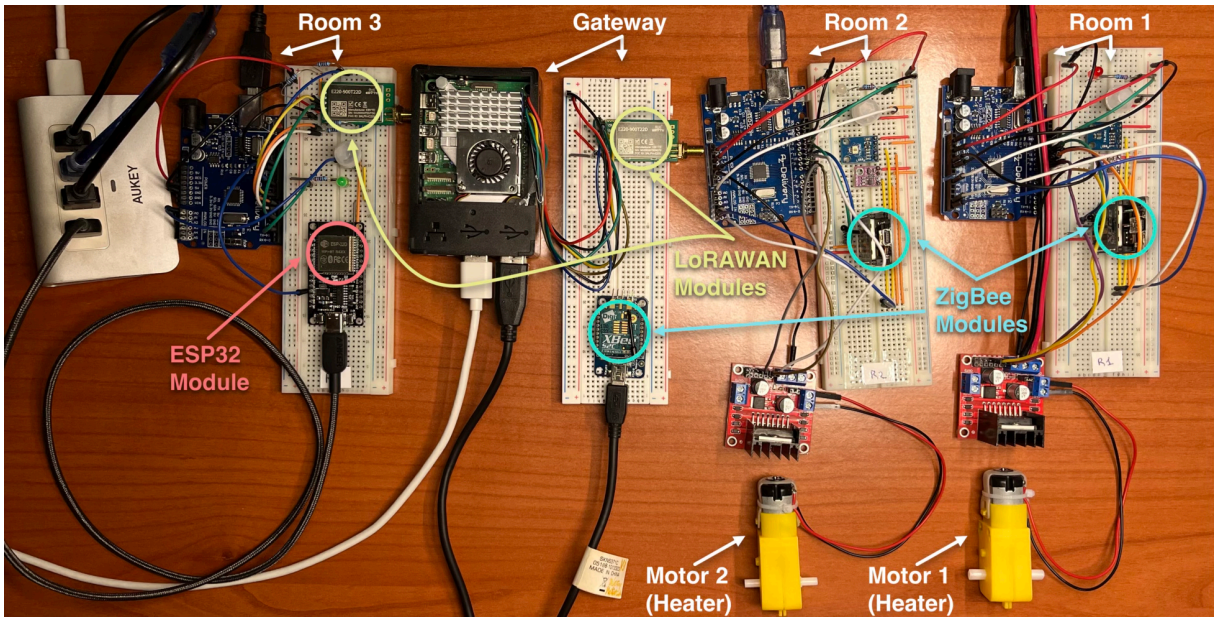


Figure 1: Physical Architecture of the Smart Home: Central Gateway and distributed Edge nodes (Room 1, Room 2, and Room 3).

As depicted in Figure 1, each room operates as an independent sensing unit, communicating environmental and motion data back to the central hub via specialized radio protocols.

2.1 Infrastructure Layout

The project simulates a smart home environment divided into two macro-areas: an indoor environment consisting of two rooms (Room 1 and Room 2), and an outdoor environment representing a separate garage (Garage or Room 3). The infrastructure is centralized around a main Gateway, ideally positioned at the center of the house. Inside each room and the garage, an Arduino microcontroller has been installed, alongside a breadboard, serving as an Edge node for interfacing with sensors, actuators, and radio communication modules.

2.2 Hardware Components, Sensors, and Actuators

To ensure comprehensive environmental monitoring, the two indoor rooms (Room 1 and Room 2) have been equipped with the following sensors:

- **PIR Sensor (AM312)[1, 2]:** Used for motion detection.
- **Luminosity Sensor (TSL2561)[3, 4]:** A light-to-digital converter that provides the illuminance parameter in Lux.

- **Environmental Sensor (BME280)[5, 6]:** A digital module used for precise sampling of temperature, humidity, and atmospheric pressure. Both the TSL2561 and BME280 sensors communicate in parallel with the Arduino using the I2C protocol via the SDA (Data) and SCL (Clock) pins.

Regarding the actuators, the system includes:

- **Lighting (LED):** In all environments (including the rooms and the garage), lighting is simulated using an LED. The LED receives digital commands from the Arduino, passing through a 220 Ω current-limiting resistor. This resistor is essential to lower the drawn current to approximately 15 mA (considering the 5V Arduino output and the LED's voltage drop), protecting the microcontroller's pins from potential overloads. Activation occurs only if the PIR detects motion and, simultaneously, the TSL2561 registers low ambient light.

- **Heating (DC Motor):** Inside the indoor rooms, the operation of a heating system (Heater) is simulated using a 5V DC motor. To drive it safely, an L298N dual H-bridge Motor Driver module was employed. The driver is powered by an external 12V source; this architectural choice is crucial to isolate the power circuit from the logic circuit, avoiding voltage drops (brownouts) or absorption peaks that could cause the Arduino board to restart or sustain damage.

3 Network Architectures and Communication Protocols

The central node of the system (Gateway) is implemented on a Raspberry Pi 5. To manage the heterogeneous network topology, two radio modules are interfaced with the Gateway: a Zigbee module (XBee S2C) connected via an "XBee to USB" adapter, and a LoRa module (EByte E220-900T22D LLCC68) [7] wired directly to the Raspberry Pi's GPIO pins, driven by the open-source library provided by Xreef [8, 9].

3.1 Zigbee Network Architecture

The Zigbee network is dedicated to communication with the indoor nodes (Room 1 and Room 2) and consists of a Coordinator node (connected to the Gateway) and two End Devices (connected to the Arduinos in the rooms). The parameterization of the modules was carried out using XCTU, the official configuration software provided by Digi International.

From a networking perspective, the three nodes communicate on the same operating channel (Channel C) and share the same PAN ID (Personal Area Network ID = 1234), thus forming a closed and secure network. At the routing level, an asymmetrical configuration was implemented:

- The **Coordinator**, as shown in Figure 1 (configured with the CE = 1 parameter, i.e., Coordinator Enable) sends commands to the rooms in *Broadcast* mode (DL parameter set to FFFF). This choice allows the Gateway to send a general command without having to maintain complex routing tables for the individual 64-bit addresses of the End Devices.

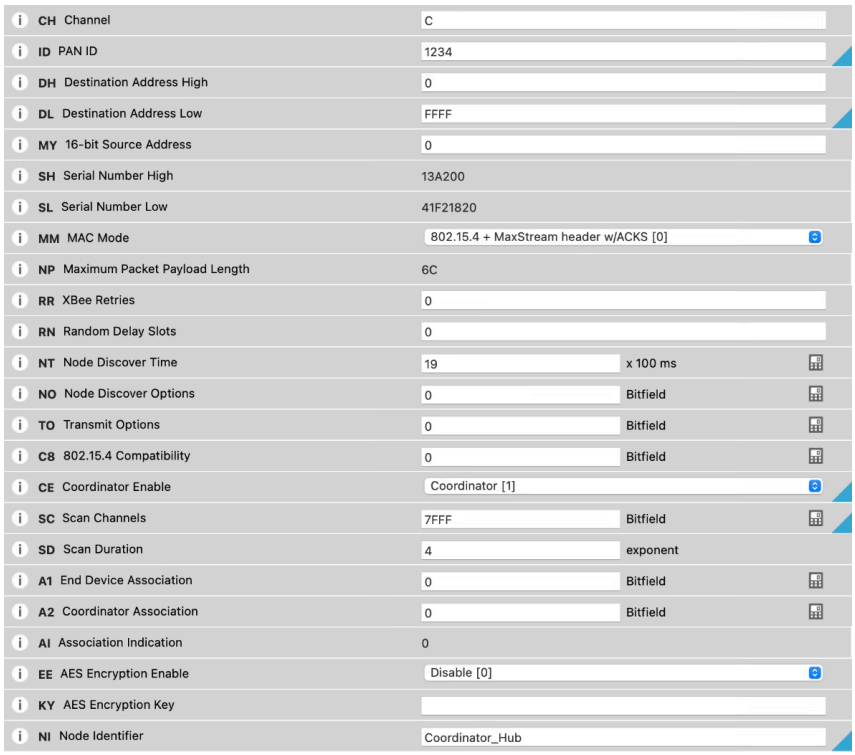


Figure 2: Configuration parameters for the Zigbee Coordinator node via XCTU.

- The **End Devices** (CE = 0), on the contrary, utilize *Direct Communication* (Point-to-Point) for data transmission. By setting the DH (Destination High) and DL (Destination Low) parameters exactly to the MAC serial address of the Coordinator (as shown in Figures 3 and 4), the sensors send telemetry directly to the Gateway. This approach drastically reduces network overhead, collisions, and broadcast noise, optimizing energy consumption.

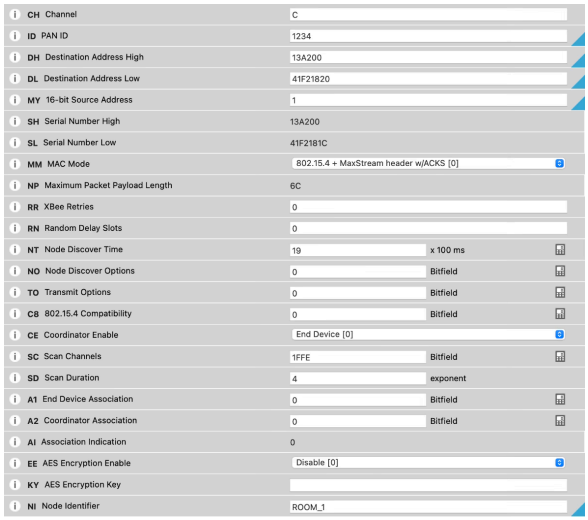


Figure 3: Configuration for Room 1.

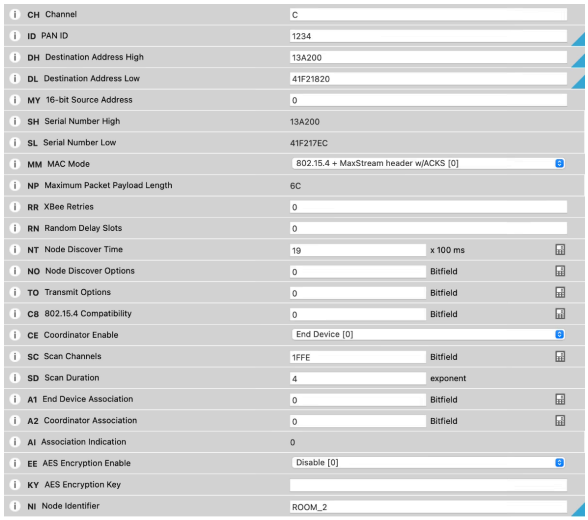


Figure 4: Configuration for Room 2.

3.2 LoRa Network Architecture

For communication with the outdoor node (Garage / Room 3), LoRa technology was chosen, ideal for overcoming physical wall obstacles and covering greater distances without signal loss. Additionally, an ESP32 module is present in the garage, leveraging the BLE (Bluetooth Low Energy) protocol to receive a PIN input from the mobile application, simulating the unlocking (Smart Lock) of the garage door.

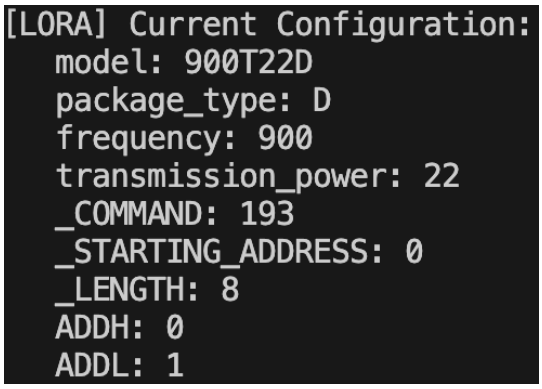


Figure 5: LoRa Gateway Configuration.

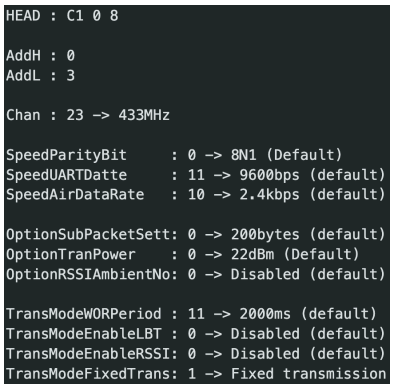


Figure 6: LoRa Room 3 Configuration.

The LoRa network is structured for Point-to-Point communication. The module connected to the Gateway has the address 01, as shown in Figure 4, while the remote node in Room 3 has the address 03, as illustrated in Figure 5. Both operate on channel 23. For the E220-900T22D module series, the base frequency is 850 MHz; operating on channel 23, the effective transmission frequency settles at 873 MHz[1]. All remaining radio parameters were left at their default values, except for the transmission mode, set to Fixed Transmission Mode = 1, which is the fundamental requirement to enable exact recipient addressing in Point-to-Point communication.

[1] *Technical Note:* During testing, the serial output of the library used to manage the EByte module might erroneously indicate the frequency as "433 MHz". This is a known hardcoding bug within the macro of some open-source libraries for Arduino, which print the default value of the 400 MHz family while ignoring the hardware's actual 900 MHz setting. Physical communication still occurs correctly on the 873 MHz spectrum.

4 Edge Nodes Firmware Development (Microcontrollers)

The Edge layer of the architecture is composed of the microcontrollers deployed across the different rooms. The firmware for these nodes was developed to prioritize local execution and non-blocking communication. By handling basic logic directly on the microcontroller, the system can react promptly to physical inputs (such as sensor triggers or local actuators) without strictly relying on the central Gateway for immediate decision-making.

4.1 Zigbee Nodes (Room 1 and Room 2)

The inside nodes (Room 1 and Room 2) share the same basic logical structure, differing only in the room identifier. Below is an analysis of how the reference source code operates.

4.1.1 Initialization and Sensor Setup

In the setup phase, the microcontroller prepares to operate by initializing the necessary libraries and establishing communication with the environmental sensors via the I2C bus (BME280 and TSL2561). Radio communication with the XBee module is managed via a `SoftwareSerial` interface on pins 10 and 11; this design precaution keeps the hardware serial port (`Serial`) free for USB debug operations.

```
SoftwareSerial Xbee(10, 11); // RX, TX Zigbee

void setup(void) {
  Serial.begin(9600);
  Xbee.begin(9600);
  stopMotors(); // Initialization in a safe state (Heater OFF)

  // I2C environmental sensors initialization
  status = bme.begin(0x76);
  if (!tsl.begin()) {
    Serial.print("TSL2561 detection error");
    while (1) delay(10);
  }

  // PIR Sensor calibration and Pin setup
  pinMode(LED_PIN, OUTPUT);
  pinMode(PIR_PIN, INPUT);
  digitalWrite(PIR_PIN, LOW);

  // Setup Motor Driver (L298N)
  pinMode(enA, OUTPUT);
  pinMode(in1, OUTPUT);
  pinMode(in2, OUTPUT);

  Xbee.println("Hello from ROOM_1 ");
}
```

4.1.2 Timed Environmental Telemetry Transmission

For the periodic transmission of environmental parameters, the use of blocking functions like `delay()` was strictly avoided. The implementation utilizes the `millis()` function to transmit data every 30 seconds; in this way, the node is free to continuously scan the motion sensors and radio directives. The data is encapsulated in a formatted text packet with separators (structure `ID_Room:Temperature:Humidity:Pressure`), optimized to facilitate parsing operations (string split) on the Gateway side.

```
// Sending Temp:Hum:Press every 30 seconds
if (millis() - count > 30000) {
  String packetTemp = ROOM_ID;
  packetTemp += ":";
  packetTemp += String(bme.readTemperature(), 1);
  packetTemp += ":";
  packetTemp += String(bme.readHumidity(), 1);
  packetTemp += ":";
  packetTemp += String((bme.readPressure() / 100.0F), 1);

  Xbee.println(packetTemp);
  count = millis();
}
```

4.1.3 Local Edge Automation (PIR + TSL2561)

The microcontroller does not delegate the decision to turn on the light to the Gateway, thus minimizing response latency. The local combinatorial logic verifies that there is motion (`PIR == HIGH`) and that the room is actually dark (`lux < 400`). If affirmative, it turns on the LED and notifies the Gateway of the physical state change by sending an update string (e.g., `Light_R1 = 1`), which is essential for keeping the user Dashboard synchronized and providing correct data to the AI model. A similar logic, based on an internal decay timer, manages the automatic turn-off when the movement ceases.

4.1.4 Reception and Parsing of Remote Commands

The firmware remains constantly listening for any commands issued by the Zigbee Coordinator. Incoming bytes are accumulated in the `inputString` buffer until the newline terminator character `\n` is detected. Once reception is complete, the `parseCommand(String command)` function is invoked, which performs the following operations:

- **Standardization:** Converts the text to lowercase and removes whitespace.
- **Filtering (Room ID):** Ensures the command is intended for the correct room (e.g., verifying the presence of "r1"). If the ID does not match, the packet is discarded.
- **Actuation and Feedback:** Recognizes the targets `heater` (acting on the L298N module pins) or `light` (on the LED), applies the command, and immediately transmits an execution feedback to the Gateway.

```
void parseCommand(String command) {
  command.trim();
  command.toLowerCase();

  if (command.indexOf("r1") == -1 && command.indexOf("room1") ==
     -1) {
    return; // Ignore if it's not for Room 1
  }

  // Heater Commands
  if (command.indexOf("heater") >= 0) {
    if (command.indexOf("off") >= 0) {
      stopMotors();
      Xbee.println("Heater_R1 = 0");  // Back command to Gateway [
         OFF]
    } else if (command.indexOf("on") >= 0) {
      digitalWrite(in1, LOW);
      digitalWrite(in2, HIGH);
      analogWrite(enA, SPEED_HIGH);
      Xbee.println("Heater_R1 = 1");  // Back command to Gateway [
         ON]
    }
  }
  // [Omitted: symmetrical logic for Light Commands]
}
```

4.2 Room 3 (Garage) Node

The remote node associated with the garage employs the LoRa protocol to easily overcome signal range limitations and wall obstacles of the outdoor environment. Unlike internal nodes, this Arduino microcontroller performs a dual task: it manages local automation (sensors and actuators) and physically interfaces with an external ESP32 module.

4.2.1 Initial Configuration and EBYTE Module Setup

This section defines the pins and crucial parameters for network reliability (QoS). In the `setup()`, in addition to preparing the sensors, the node queries the LoRa module to obtain its configuration, ensuring the radio communication is ready before starting the system. A best practice implemented in this phase involves clearing the serial buffers (`Serial.flush()` and `mySerial.flush()`) before entering the main lifecycle of the firmware; this operation is essential to prevent past or corrupted data from interfering with new incoming messages.

```
#include <LoRa_E220.h>

// QoS CONFIGURATION
#define MAX_RETRIES 10      // Maximum number of attempts
#define ACK_TIMEOUT 2000    // ACK wait time in milliseconds

SoftwareSerial mySerial(8, 10);
LoRa_E220 e220ttl(&mySerial, 4, 7, 6);
void setup() {
  Serial.begin(9600);

  // LORAWAN Initialization
  e220ttl.begin();

  // QoS MODIFICATION: Send with retry
  sendFixedMessageWithRetry(0, 1, 23, "Hello from ROOM_3 ");

  Serial.flush(); // Ensures all pending debug messages are fully
     transmitted via USB before proceeding
  mySerial.flush(); // Clears the communication buffer
}
```

4.2.2 Command Reception and ACK Management

In this block of the `loop()`, the node listens for incoming messages. It recognizes commands intended for Room 3 (turning lights on/off) and responds using the new QoS function to confirm execution. Furthermore, it intercepts ACKs (Acknowledgements) arriving late to prevent logical overlaps.

```
if (e220ttl.available() > 0) {
  delay(100);
  ResponseContainer rc = e220ttl.receiveMessage();
  if (rc.status.code == 1) {
    String command = rc.data;
    command.toLowerCase();
    // If it's an ACK, ignored in the main loop
        if (command.indexOf("ack") >= 0) {
      Serial.println("ACK received outside cycle (late?)");
    }
    if (command.indexOf("r3") || command.indexOf("room3") >= 0 ) {
      if (command.indexOf("light") >= 0) {
        if (command.indexOf("off") >= 0) {
          digitalWrite(LED_PIN, LOW);
          sendFixedMessageWithRetry(0, 1, 23, "Light_R3 = 0"); //
             Back command to Gateway [OFF]
        } else if (command.indexOf("on") >= 0) {
          digitalWrite(LED_PIN, HIGH);
          sendFixedMessageWithRetry(0, 1, 23, "Light_R3 = 1");} //
             Back command to Gateway [ON]
      }
    }
```

4.2.3 Local Events: ESP32 (BLE) and PIR Sensor Integration

The node is not just a receiver but acts actively based on the environment with a signal coming from an ESP32. When the Arduino detects a rising edge (transition from LOW to HIGH) on that pin, it recognizes the successful BLE authentication by the user. In response, it generates and sends an unlock packet (containing the code) to the Gateway via the LoRa network.

```
currentESPstate = digitalRead(ESP_PIN);
if (currentESPstate == HIGH && lastESPstate == LOW) {
  Serial.println("Received from ESP32 the code");
  sendFixedMessageWithRetry(0, 1, 23, "Code received");
  delay(500);
}
lastESPstate = currentESPstate;
```

4.2.4 The QoS (Quality of Service) Mechanism

This application-level mechanism solves the main problem of long-range wireless networks: packet loss. This function sends a message and waits for a maximum time of `ACK_TIMEOUT`. If it receives the string "ack", the transmission is confirmed as complete. Otherwise, it retries up to a maximum of `MAX_RETRIES`, ensuring a very high probability that the vital data reaches its destination.

```
void sendFixedMessageWithRetry(byte ADDH, byte ADDL, byte CHAN,
    String message) {
  bool ackReceived = false;
  int attempt = 1;
  while (attempt <= MAX_RETRIES && !ackReceived) {
    ResponseStatus rs = e220ttl.sendFixedMessage(ADDH, ADDL, CHAN,
        message);
    // Wait for ACK
    unsigned long startWait = millis();
    while (millis() - startWait < ACK_TIMEOUT) {
      if (e220ttl.available() > 1) { // Check for incoming response
        ResponseContainer rc = e220ttl.receiveMessage();
        String response = rc.data;
        response.toLowerCase();
        // Check if the received message contains "ack"
        if (response.indexOf("ack") >= 0) {
          ackReceived = true;
          Serial.println(">> ACK RECEIVED! Transmission OK.");
          break; // Exit the wait loop
        }
      }
    }
    if (!ackReceived) {
      attempt++;
      if (attempt <= MAX_RETRIES) {
        delay(500); // Brief pause before next attempt
```

4.3 BLE Access Control System (ESP32 Module)

To implement the "Smart Lock" functionality for the garage, an ESP32 module was introduced. This component exposes a Bluetooth Low Energy (BLE) server designed to interface with the mobile application developed in parallel.

4.3.1 BLE Definitions and Event Management (Callbacks)

In this first part, the UUIDs (Universally Unique Identifiers) necessary to create the BLE Service and Characteristic are defined. Additionally, Callback classes are implemented: these are fundamental because they allow the ESP32 to handle connections and data reception asynchronously, reacting to events (like a smartphone connecting or sending data) without blocking the microcontroller.

```
#define SERVICE_UUID "4fafc201-1fb5-459e-8fcc-c5c9c331914b"
#define CHARACTERISTIC_UUID "beb5483e-36e1-4688-b7f5-ea07361b26a8"

BLEServer *pServer = NULL;
BLECharacteristic *pCharacteristic = NULL;
bool deviceConnected = false;
bool oldDeviceConnected = false;
bool dataReceived = false;
String receivedData = "";
String CODE = "1234";

// SERVER CALLBACKS
class MyServerCallbacks : public BLEServerCallbacks {
  void onConnect(BLEServer *pServer) {
    deviceConnected = true;
  };
  void onDisconnect(BLEServer *pServer) {
    deviceConnected = false;
  }
};

// CHARACTERISTIC CALLBACKS
class MyCallbacks : public BLECharacteristicCallbacks {
  void onWrite(BLECharacteristic *pCharacteristic) {
    String rxValue = pCharacteristic->getValue();
    if (rxValue.length() > 0) {
      receivedData = rxValue;
      dataReceived = true;
      receivedData.trim();
    }
  }
};
```

4.3.2 BLE Server Setup and Advertising

This section shows the step-by-step configuration of the BLE peripheral. The ESP32 is initialized with the name "ESP32-Room3(Gate)" to make it visible and recognizable by

smartphones during scanning. The server, the service, and the characteristic with the necessary permissions (Read, Write, Notify, and Indicate) are then instantiated. Finally, the ESP32 begins to broadcast its presence by activating Advertising.

```
void setup() {
  Serial.begin(115200);
  // Create the BLE Device
  BLEDevice::init("ESP32-Room3(Gate)");

  // Create the BLE Server
  pServer = BLEDevice::createServer();
  pServer->setCallbacks(new MyServerCallbacks());

  // Create the BLE Service
  BLEService *pService = pServer->createService(SERVICE_UUID);

  // Create a BLE Characteristic
  pCharacteristic = pService->createCharacteristic(
                      CHARACTERISTIC_UUID,
                      BLECharacteristic::PROPERTY_READ |
                      BLECharacteristic::PROPERTY_WRITE |
                      BLECharacteristic::PROPERTY_NOTIFY |
                      BLECharacteristic::PROPERTY_INDICATE);

  pCharacteristic->addDescriptor(new BLE2902());
  pCharacteristic->setCallbacks(new MyCallbacks());

  // Start the service
  pService->start();

  // Start advertising
  BLEAdvertising *pAdvertising = BLEDevice::getAdvertising();
  pAdvertising->addServiceUUID(SERVICE_UUID);
  pAdvertising->setScanResponse(false);
  BLEDevice::startAdvertising();
  Serial.println("BLE Smart Gate Ready...");
}
```

4.3.3 Authentication and Signaling Logic

When data arrives via Bluetooth, the ESP32 verifies if the received PIN matches the saved one (in our case `CODE = "1234"`). If the PIN is correct, it raises the logical level of the `ARDUINO_PIN`, physically sending the impulse to the connected Arduino. If it is wrong, it notifies the BLE client of the error. Furthermore, it intelligently handles disconnections by reactivating Advertising to allow new devices to connect in the future.

```
void loop() {
  if (dataReceived) {
    if(receivedData == CODE){
      Serial.println("Correct PIN");
      digitalWrite(ARDUINO_PIN, HIGH);
      delay(100);
    } else {
      Serial.println("Wrong PIN");
      pCharacteristic->setValue("ERROR");
      pCharacteristic->notify();
    }
    digitalWrite(ARDUINO_PIN, LOW);
    dataReceived = false;
    receivedData = "";
  }

  // disconnecting
  if (!deviceConnected && oldDeviceConnected) {
    delay(500);                    // Give the bluetooth stack the
       chance to get things ready
    pServer->startAdvertising();  // Restart advertising
    Serial.println("Restarting Advertising ...");
    oldDeviceConnected = deviceConnected;
  }

  // connecting
  if (deviceConnected && !oldDeviceConnected) {
    oldDeviceConnected = deviceConnected;
  }
  delay(50);  //Small delay for connection stability
}
```

5 Software Architecture of the Central Gateway (Raspberry Pi)

The architecture of the central node (Gateway) has been designed following a strict modular paradigm. This engineering choice ensures high scalability and allows asynchronous management of hardware communication (via dedicated threads), Artificial Intelligence (AI) processing, and routing MQTT messages to the user dashboard.

5.1 Modular Paradigm and Data Abstraction

To keep the code clean and maintainable, the supporting logic was separated into specialized modules, strictly separating the translation layer from the control layer.

- **The `protocol_router.py` module:** Acts as the central dispatcher. Its task is to receive a generic JSON command intended for a specific room (e.g., `{"light": "on"}` for `room1`), identify the associated hardware protocol via a static configuration map, and translate it into a raw string compatible with the Arduino microcontrollers' firmware.

```
ROOM_PROTOCOL_MAP = {
    "room1": "zigbee",
    "room2": "zigbee",
    "room3": "lora",
    "garage": "ble"
}

def translate_payload_for_arduino(room, payload):
    prefix = ""
    if room == "room1": prefix = "r1"
    elif room == "room2": prefix = "r2"
    elif room == "room3": prefix = "r3"
    else: return str(payload)

    if isinstance(payload, dict):
        device = list(payload.keys())[0].lower()
        action = list(payload.values())[0].lower()
        return f"{prefix} {device} {action}"

    return f"{prefix} {str(payload).lower().strip()}"
```

- **The `ai_features.py` module:** Represents the abstraction layer for the Machine Learning model. It takes the raw state of the house (environmental data and relay status) and maps it into the exact 21 features required by the LSTM neural network. In addition to direct mapping, this module is responsible for inferring the energy loads of household appliances (e.g., estimating microwave use based on the time and motion sensor) and introducing statistical noise to simulate normal power grid fluctuations, making the data highly realistic.

```
    # Microwave Logic:
    # IF motion is detected in Room 1 (Kitchen) AND it is "meal
       time",
    # THEN assume microwave is running (1.2 kW). Otherwise 0.
    if motion_r1 and (11 <= hour <= 14 or 18 <= hour <= 21):
        microwave = 1.2
    else:
        microwave = 0.0

    # Heater / Furnace Logic:
    # If the heater relay is ON, add the load of the furnace.
    furnace_1 = 1.5 if heater_r1 else 0.0
    furnace_2 = 1.0 if heater_r2 else 0.0
```

5.2 The Core Engine (gateway_smart_home.py)

The main file represents the "brain" of the entire infrastructure, acting as a bidirectional bridge between the physical sensors, the MQTT broker, and the predictive Artificial Intelligence model.

5.2.1 Global State Management (Single Source of Truth)

To avoid temporal inconsistencies between what the AI calculates, what the user sees, and the physical state of the devices, the system relies on a global dictionary called `home_state`. This dictionary is constantly updated in real-time by incoming MQTT messages (leveraging the Single Source of Truth paradigm) and acts as an instantaneous and authoritative "snapshot" of the entire home.

```
home_state = {
    "room1": {"temperature": 22.0, "humidity": 50, "pressure":
       1012, "light": "off", "heater": "off", "motion": "NOMOTION"
       },
    "room2": {"temperature": 21.0, "humidity": 45, "pressure":
       1010, "light": "off", "heater": "off"},
    # ...
}
```

5.2.2 Predictive Model and Smart Metering Integration

The Gateway's innovation lies in the thread dedicated to Artificial Intelligence (`ai_energy_thread`). This background process samples the home's state and accumulates data in a buffer. Since the LSTM model requires a temporal sequence to make a prediction, the buffer stores 60 instances (representing one minute of history). The prediction feeds a Smart Metering logic (intelligent circuit breaker): if the AI predicts an overload (e.g., `> 2.5 kW`), the system does not disconnect loads instantaneously, but starts a 60-second tolerance timer, emulating the thermal trip curve of real meters. Once the threshold is exceeded, the system autonomously disconnects the largest non-priority load.

```
                if len(buffer) == SEQUENCE_LENGTH:
                  if pred_real > MAX_HOUSE_KW:
                      overload_counter += 1
                      print(f"WARNING: Load > {MAX_HOUSE_KW} kW. Trip
                          in {60 - overload_counter} seconds.")

                      if overload_counter >= 60:
                          safe_notify("METER_TRIP", "CRITICAL", "Main
                              Breaker Warning", f"Sustained overload:
                              {pred_real:.2f} kW. Initiating
                             emergency cutoff.")

                          if AUTO_MODE:
                              device = choose_device_to_turn_off(raw)
                              if device == "heater":
                                  target_room =
                                      choose_room_for_heater()
                                  # Execute cut-off. Imposes a strict
                                       60-second cooldown
                                  safe_auto_action("EMERGENCY_CUTOFF"
                                      , target_room, {"heater": "off"
                                     }, custom_cooldown=60)

                          overload_counter = 0
                  else:
                      if overload_counter > 0:
                          clear_state("METER_TRIP")
                      overload_counter = 0
```

5.2.3 Smart Alert System (Priority and Cooldown)

To protect the user experience and avoid alert fatigue, a smart alarm management system was implemented. The `safe_notify` function applies two fundamental filters: a Priority Filter (a critical alarm suppresses lower-level alarms) and a Cooldown Timer (which prevents sending the same alert before 120 seconds have elapsed).

```
def safe_notify(rule_id, level, title, msg):
    global current_alert_level
    now = time.time()
    rule_level = ALERT_LEVELS.get(level, 1)
    # Global Priority Check
    if rule_level < current_alert_level:
        return
    # Individual Cooldown Check
    if alert_state.get(rule_id) == "active":
        if rule_id in last_alert_time and (now - last_alert_time[
            rule_id] < ALERT_COOLDOWN):
            return
    current_alert_level = rule_level
    last_alert_time[rule_id] = now
    alert_state[rule_id] = "active"
```

5.3 Physical Layer Drivers (Zigbee and LoRa)

The `zigbee_driver.py` and `lora_driver.py` modules are the software components designated for hardware interfacing. To ensure asynchronicity, both drivers start their own Continuous Read Thread (`read_loop`) upon initialization. In this way, I/O operations on the serial port do not cause bottlenecks to the Gateway's lifecycle and the execution of the neural model.

5.3.1 Data Parsing and Extraction (Zigbee Driver)

The Zigbee listening task decodes the serial packets, converting them to the JSON standard for publication on the MQTT broker. The parser effectively distinguishes complex data, splitting the string (like temperature and pressure separated by colons), from simple actuator state changes.

```
def handle_incoming(data):
    try:
        # Parsing ambient data (Format: R1:22.5:55.0:1012.0)
        if ":" in data and data.startswith("R"):
            parts = data.split(":")
            if len(parts) >= 4:
                payload = {
                    "room": parts[0].replace("R", "room"),
                    "temperature": float(parts[1]),
                    "humidity": float(parts[2]),
                    "pressure": float(parts[3])
                }
                mqtt_client.publish(TOPIC_STATUS, json.dumps(
                    payload))

        # Parsing actuator state
        elif "Light" in data:
            target_room = "room1" if "R1" in data else "room2"
            state = "on" if data.split(" = ")[1].strip() == "1"
                else "off"
            payload = {"room": target_room, "light": state}
            mqtt_client.publish(TOPIC_STATUS, json.dumps(payload))

    except (ValueError, IndexError) as e:
        print(f"[ZIGBEE PARSE ERROR] {e}")
```

5.3.2 Bidirectional Reliability (LoRa Driver)

Mirroring the Arduino C++ code, the Python module for the LoRa protocol natively implements a Retry/ACK system. Using the `threading.Event` synchronization primitive, the sending thread suspends itself, waiting for the reading thread to intercept the correct confirmation string. If the event is not unblocked within the timeout, the library automatically attempts retransmission.

```
def real_serial_send_with_retry(room, payload):
    # ... [Omitted parameter retrieval] ...

    while attempt <= max_retries and not sent_successfully:
        ack_received_event.clear()
        try:
            code = lora_module.send_fixed_message(room_3["ADDH"],
                room_3["ADDL"], room_3["CHAN"], command_string)

            if code == ResponseStatusCode.SUCCESS:
                # Blocks current thread until ACK triggers the
                    event in the read_loop
                if ack_received_event.wait(timeout=2.0):
                    sent_successfully = True

        except Exception as e:
            print(f"[LORA SEND ERROR] {e}")

        if not sent_successfully:
            attempt += 1
            time.sleep(0.5)
```

6 Data Materials and Selection

The empirical foundation of this study is built upon the "Smart Home Dataset with Weather Information" [10], sourced from a Kaggle open-source repository. This dataset was selected for its high-fidelity representation of residential energy dynamics, offering a robust framework for evaluating machine learning architectures in complex IoT environments.

6.1 Rationale for Dataset Selection

The choice of this specific dataset is predicated on several critical technical attributes that facilitate advanced predictive modeling:

- **Dual-Contextual Integration:** A primary limitation of conventional energy datasets is the isolation of consumption metrics from external variables. This dataset mitigates this by merging human-centric behavioral data (appliance-specific power draw) with synchronized environmental parameters (e.g., temperature, humidity, and wind speed).
- **Temporal Granularity:** Featuring a 1-minute sampling interval over a duration of 350 days, the dataset provides the high-resolution data necessary for capturing transient energy events. This granularity allows for the identification of micro-patterns, such as the instantaneous power spikes associated with microwave usage or the low-frequency cyclic oscillations of refrigeration units.

6.2 Feature Engineering and Dimensionality Reduction

To enhance model interpretability and computational efficiency, a rigorous feature selection process was employed. The raw dataset, comprising 32 initial variables, was strategically condensed into 11 core features within the processed `HomeC_edited.csv` file. This reduction serves to minimize stochastic noise, mitigate the risk of overfitting, and address potential issues of multicollinearity.

6.2.1 Systematic Exclusion Criteria

The following variables were excluded based on statistical and domain-specific rationales:

- **Redundant Target and Generation Metrics:** Variables such as `use [kW]`, `Gen [kW]`, and `Solar [kW]` were removed. `Use[kW]` was found to be functionally identical to `House overall [kW]`, presenting a data leakage risk. Furthermore, power generation metrics (e.g., Solar) were deemed outside the scope of this study, which focuses exclusively on mapping consumption patterns to environmental and behavioral drivers.
- **Sparse and High-Variance Appliance Data:** Consumption data for niche areas - including the Wine cellar, Barn, Well, Garage door, and Home office - were omitted. These features exhibited high sparsity (predominantly zero values) or negligible power draw, offering minimal predictive value while increasing the feature space's complexity.

- **Multicollinear Meteorological Variables:** To prevent the "curse of dimensionality" and ensure stable coefficient estimation, highly correlated weather features were pruned. Specifically, `apparentTemperature` and `dewPoint` were removed due to their high collinearity with the primary temperature variable.
- **Categorical and Qualitative Indicators:** Qualitative descriptors, such as `icon` and `summary` (e.g., "Partly Cloudy"), were discarded. These categorical labels are redundant when continuous numerical predictors like humidity, pressure, and temperature are already present to mathematically define the atmospheric state.
- **Low-Correlation Environmental Factors:** Statistical preliminary analysis indicated that factors such as `visibility`, `windBearing`, and `precipProbability` shared a negligible correlation with indoor appliance activation. Their removal allows the model to prioritize the primary thermal and hygroscopic drivers of energy demand.

The rationale for these exclusions is quantitatively supported by the Pearson correlation matrix (see Figure 7), which illustrates the linear dependencies between variables. High coefficients between features like 'Use' and 'House Overall' ($r = 1.00$) confirm the presence of redundant data structures, while the low correlation coefficients observed for niche appliances justify their removal to streamline the feature space.

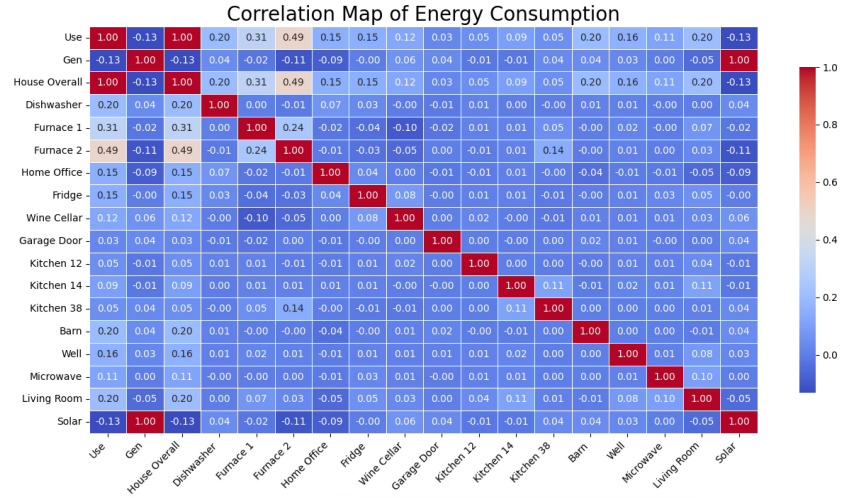


Figure 7: Heatmap of the Pearson correlation coefficients for the energy consumption dataset.

6.3 Data Preprocessing and Signal Conditioning

Raw IoT telemetry is inherently stochastic and often contains artifacts that can degrade predictive accuracy. A multi-stage preprocessing pipeline was implemented to ensure data quality and model stability.

6.3.1 Missing Value Analysis and Data Integrity

Scanning 503,911 observations revealed a single row with null values. Given the 1-minute temporal resolution, this record was excluded rather than imputed (e.g., via mean interpolation) to preserve true variance without introducing synthetic bias.

6.3.2 Temporal Feature Engineering

Since ISO-standardized timestamps are computationally opaque to regression algorithms, the following cyclical features were extracted to capture human behavior:

- **Diurnal Patterns (Hour):** Capturing peak demand periods, such as evening cooking or morning routines.
- **Weekly/Monthly Trends (Day, Month):** Accounting for weekend behavior shifts and seasonal HVAC fluctuations.

By decomposing the timestamp, the model can effectively map energy consumption to the "human clock", significantly improving its ability to forecast periodic load spikes.

6.3.3 Outlier Mitigation and Winsorization

Smart meter data is prone to "transient spikes" from motor startups or inductive loads. To prevent these from skewing the model's weights, a Winsorization technique was applied.

- **Thresholding:** Values in the `house_overall` feature were capped at the 99th percentile (6.47kW).
- **Impact:** Approximately 10,076 extreme values were adjusted.

This process ensures that the model learns the "steady-state" behavior of the home rather than over-adjusting to non-representative electrical noise.

6.3.4 Non-Gaussian Scaling via RobustScaler

Residential energy data typically exhibits a heavy right tail. Since traditional Z-score standardization is hypersensitive to outliers, the RobustScaler was utilized:

$$X_{scaled} = \frac{X - Q_2(X)}{Q_3(X) - Q_1(X)} \tag{1}$$

By utilizing the Median (Q_2) and the Interquartile Range (IQR: $Q_3 - Q_1$), the features were scaled to a uniform magnitude while remaining inherently resistant to the influence of extreme values. This ensures that the gradient descent process remains stable and converges more efficiently.

6.4 Model Architecture and Training Strategy

The core of the predictive framework is a specialized Deep Learning architecture designed to handle the non-linear, sequential nature of IoT energy telemetry. While standard Feedforward Neural Networks (FNN) are effective for static data, they fail to account for the temporal "memory" inherent in residential power consumption.

6.4.1 The Rationale for Bidirectional LSTM

Energy consumption is fundamentally a time-series problem where the load at time t depends on $t-1$. To capture these dynamics and mitigate the vanishing gradient problem, we implemented a Long Short-Term Memory (LSTM) network. We utilized a Bidirectional LSTM (Bi-LSTM) wrapper for two main advantages:

- **Temporal Contextualization:** Unlike unidirectional LSTMs that only process information from $t-n$ to t, a Bi-LSTM processes the input sequence in two directions simultaneously (forward and backward). This allows the hidden layer to aggregate information from both past and "future" states within a specific look-back window.
- **Symmetrical Pattern Recognition:** Household appliances often exhibit symmetrical power signatures - such as the distinct ramp-up and cool-down phases of a HVAC compressor or a furnace cycle. By processing the sequence in both directions, the model gains a holistic understanding of these energy events, leading to a more robust representation of the underlying load profile.

6.4.2 Structural Configuration

The model's depth was carefully calibrated to balance representational power with computational efficiency. The architecture comprises:

- **Input Layer:** Accepting the 11 preprocessed features across a defined temporal window.
- **Bi-LSTM Layers:** Utilizing two stacked bidirectional layers to extract deep hierarchical temporal features.
- **Dropout Regularization**.
- **Dense Output Layer**.

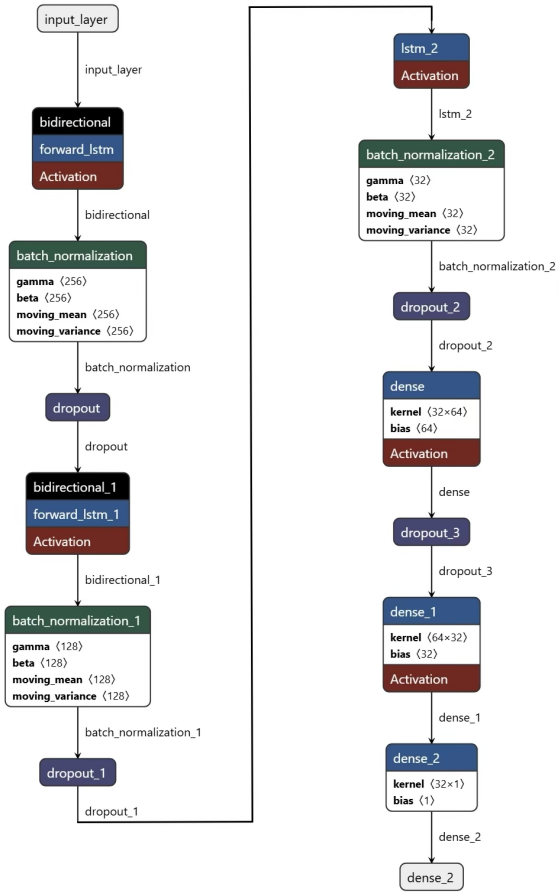


Figure 8: Model Architecture.

6.5 Detailed Architectural Configuration

The predictive engine, encapsulated in the `build_advanced_lstm_model` function, is a deep, regularized Recurrent Neural Network. The architecture follows a hierarchical design, transitioning from high-dimensional temporal extraction to low-dimensional regression.

6.5.1 Hierarchical Feature Extraction

The model utilizes a stacked recurrent approach to decompose the complexities of IoT telemetry:

- **Primary Bidirectional LSTM Layers (128 & 64 units):** These layers serve as the foundational feature extractors. By processing sequences of weather and appliance data over the defined `SEQUENCE_LENGTH`, they identify multi-scale temporal dependencies. The first layer utilizes `return_sequences=True`, ensuring that the entire temporal manifold is passed to the subsequent layer. This allows the network to learn hierarchical representations of time-series data, from micro-fluctuations to macro-trends.
- **Convergent LSTM Layer (32 units):** The final recurrent stage aggregates the rich contextual features from the bidirectional layers into a high-density vector. By omitting the sequence return at this stage, the model effectively "summarizes" the entire look-back window into a fixed-length latent representation.

6.5.2 Regularization and Optimization Components

To ensure the model remains robust against the inherent volatility of residential energy data, several optimization layers were integrated:

- **Batch Normalization:** Implemented after each LSTM block, BN addresses the challenge of Internal Covariate Shift. IoT data often causes dramatic shifts in internal weight distributions during backpropagation; BN standardizes these outputs, significantly accelerating convergence and stabilizing the gradient flow.
- **Stochastic Dropout (0.2 – 0.3):** Given that smart home data is frequently punctuated by "human noise" - such as stochastic, short-duration appliance usage - Dropout layers were utilized to prevent overfitting. By randomly deactivating a percentage of neurons during the training phase, the model is forced to prioritize robust, generalizable patterns (e.g., thermal lag) over the memorization of anomalous spikes.

6.5.3 Multi-Layer Perceptron (MLP) Decoder

The "decision-making" core of the network consists of a series of fully connected Dense Layers (64, 32, 1). These layers perform the final non-linear mapping, translating the abstract temporal features extracted by the LSTMs into a singular, continuous scalar value: the predicted `House overall [kW]`.

6.6 Stochastic Optimization and Training Dynamics

The convergence of deep recurrent architectures is highly sensitive to the optimization landscape. To ensure robust parameter estimation, we implemented an adaptive training strategy centered on the Adam (Adaptive Moment Estimation) optimizer.

6.6.1 Adaptive Gradient Descent

The Adam optimizer was selected due to its ability to compute individual adaptive learning rates for different parameters from estimates of first and second moments of the gradients. An initial learning rate of 0.001 was established, providing an optimal balance between rapid initial convergence and the avoidance of catastrophic divergence during the early epochs of the training cycle.

6.6.2 Precision Tuning via Learning Rate Decay

To navigate the complex loss surfaces typical of high-dimensional energy data, we integrated a `ReduceLROnPlateau` callback mechanism. Energy models frequently encounter "plateaus" where the validation loss (`val_loss`) ceases to improve, often indicating that the model is oscillating around a local minimum.

- **Trigger Mechanism:** The system monitors the validation loss over a patience window of 5 epochs.
- **Adjustment:** Upon detection of a stall in improvement, the learning rate is decayed by a factor of 0.5 (50%).

This "annealing" process allows the model to take progressively smaller steps, enabling it to settle into a more precise, globally optimal solution.

6.6.3 Numerical Stability and Gradient Clipping

LSTMs are mathematically predisposed to the Exploding Gradient phenomenon, where large error gradients accumulate and result in massive updates to network weights, leading to numerical instability.

- **Gradient Clipping (`clipnorm=1.0`):** To mitigate this, we implemented a norm-based gradient clipping strategy. By capping the gradient norm at 1.0, we ensure that the optimization process remains stable even when the model encounters anomalous energy spikes or transient noise in the input sequence.

6.7 Objective Functions and Performance Metrics

For residential energy forecasting, where data is frequently punctuated by stochastic spikes, a robust objective function is required to maintain structural stability.

6.7.1 The Rationale for Huber Loss

While Mean Squared Error (MSE) is the standard for regression, its quadratic penalty for large residuals makes it hypersensitive to outliers. In the context of IoT smart meter data - where instantaneous motor startups or "noise" can create massive, split-second power

spikes - MSE would force the model to skew its global predictions upward to compensate for these anomalies. To mitigate this, we implemented Huber Loss, a hybrid objective function defined as:

$$L_\delta(a) = \begin{cases} \frac{1}{2}a^2 & \text{if } |a| \leq \delta \\ \delta \cdot (|a| - \frac{1}{2}\delta) & \text{otherwise} \end{cases} \tag{2}$$

- **Adaptive Sensitivity:** Huber loss functions quadratically (like MSE) for small errors, ensuring a smooth gradient near the target. However, for errors exceeding the threshold δ, it transitions to a linear penalty (like Mean Absolute Error).
- **Outlier Resilience:** Synergizing with the RobustScaler used in the preprocessing phase, Huber Loss ensures the network learns the baseline consumption profile of the residence rather than "chasing" non-representative electrical transients.

6.7.2 Multidimensional Evaluation Metrics

To provide a comprehensive assessment of the model's predictive capabilities, we monitored a diverse suite of statistical indicators across the training and validation sets:

- **Loss & Validation Loss (`val_loss`):** The primary Huber loss was tracked across the training data and a 15% held-out validation split. A divergence between these two metrics serves as a diagnostic for overfitting. An `EarlyStopping` callback was integrated to terminate the training process if the `val_loss` failed to improve, preserving the model's generalizability.
- **Mean Absolute Error (MAE):** This provides a highly interpretable, human-readable metric, representing the average magnitude of the error in scaled kilowatts [kW].
- **Coefficient of Determination (R^2 Score):** Calculated during the final testing phase, R^2 quantifies the proportion of variance in energy consumption that is successfully explained by the meteorological and appliance-level predictors. A score approaching 1.0 signifies a high-fidelity model.
- **Mean Absolute Percentage Error (MAPE):** To facilitate communication with non-technical stakeholders, MAPE expresses the model's error as a relative percentage. This allows for intuitive accuracy statements, such as "the model predicts total household load within a $\pm X\%$ margin of error".

6.8 Results and Discussion

The predictive performance of the Bidirectional LSTM model was evaluated on a held-out test set to determine its efficacy in capturing the volatile dynamics of residential energy consumption.

6.8.1 Global Model Performance

The model achieved a Coefficient of Determination (R^2) of 0.7317, indicating that approximately 73.17% of the variance in household energy consumption is successfully explained

by the integrated appliance and weather features. As illustrated in Figure 9, the predicted consumption (orange) closely tracks the actual energy demand (blue) throughout the entire time series.

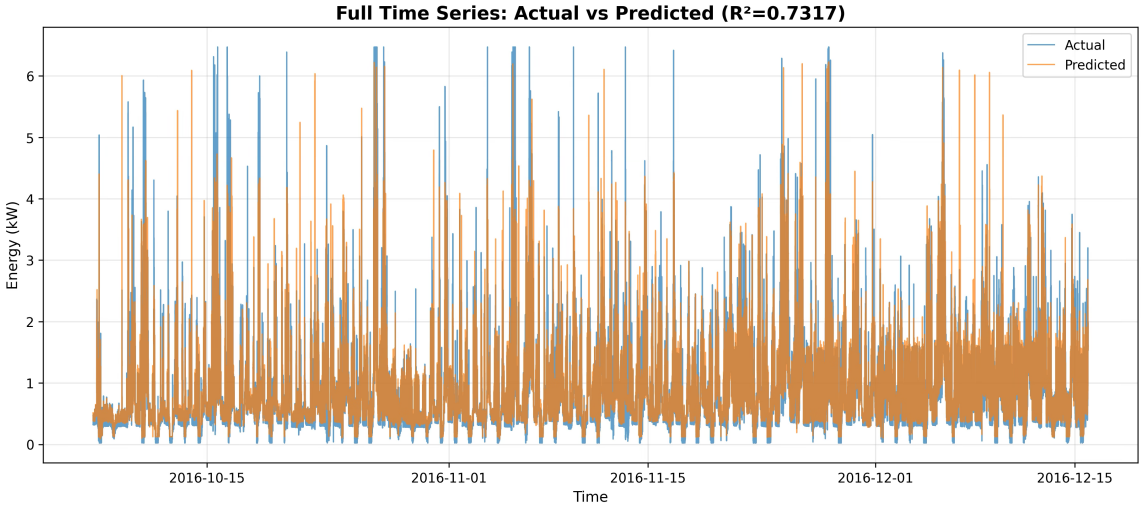


Figure 9: Full Time Series analysis showing the alignment between predicted and actual energy consumption.

6.8.2 Temporal Pattern Recognition

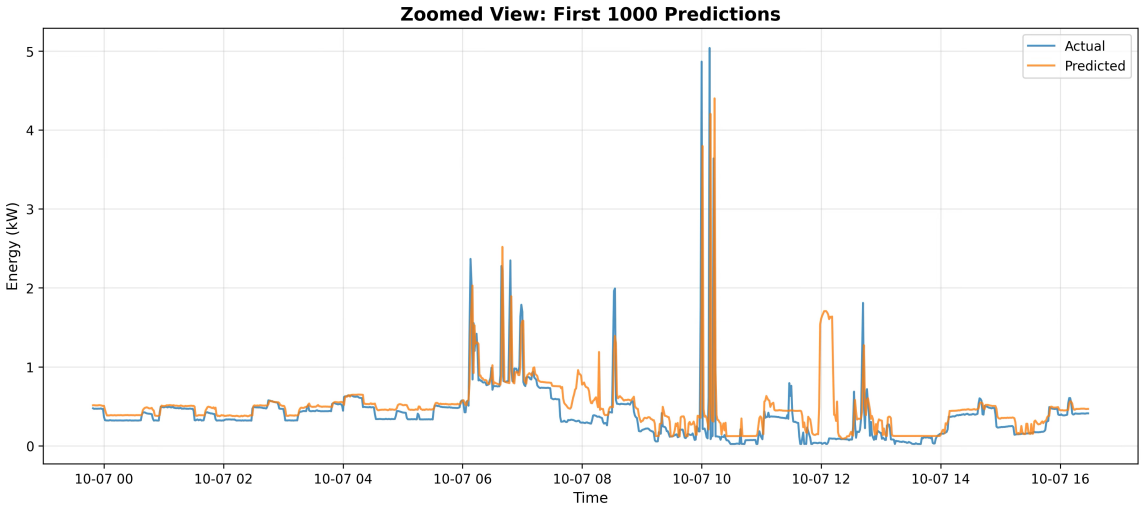


Figure 10: Zoomed View of the first 1,000 predictions, highlighting the model's ability to track micro-patterns in energy usage.

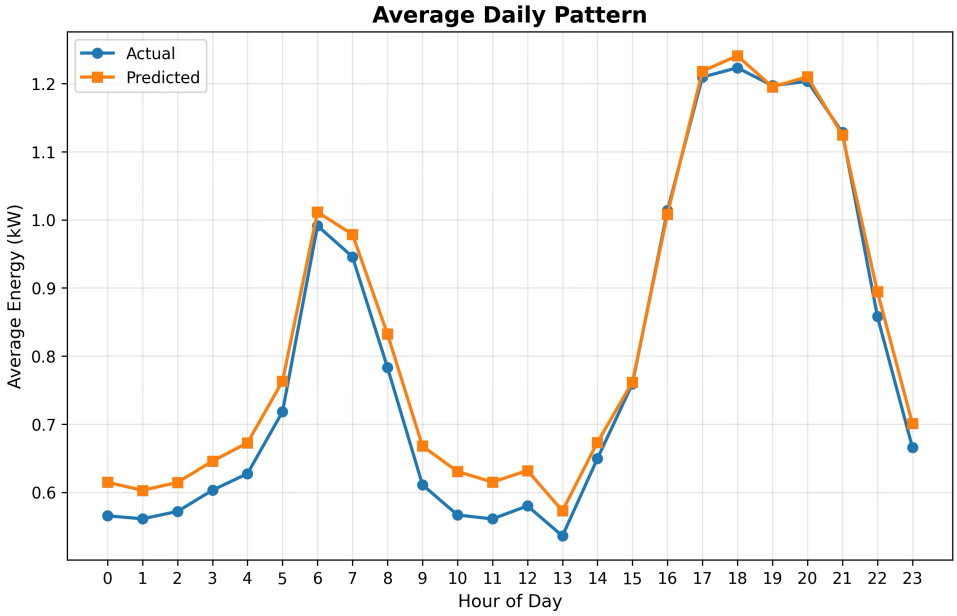


Figure 11: Comparison of average daily consumption patterns, illustrating the model's capture of diurnal human behavior.

6.8.3 Error and Residual Analysis

To assess the model's reliability, a scatter plot of Predicted vs. Actual values was generated (Figure 12). The dense clustering of data points along the 45° "Perfect Prediction" line confirms a strong linear relationship with minimal systemic bias.

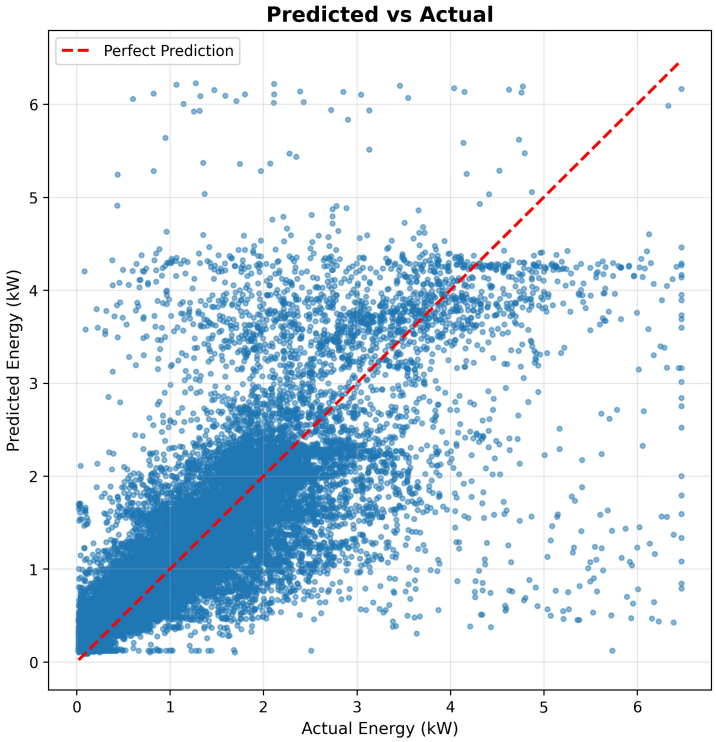


Figure 12: Regression scatter plot illustrating the correlation between actual and predicted kW values.

The distribution of residuals (Figure 13) exhibits a sharp Gaussian-like peak centered near zero. The lack of significant skewness in the error histogram suggests that the Huber Loss and RobustScaler successfully mitigated the influence of outliers, preventing extreme spikes from degrading the model's overall accuracy.

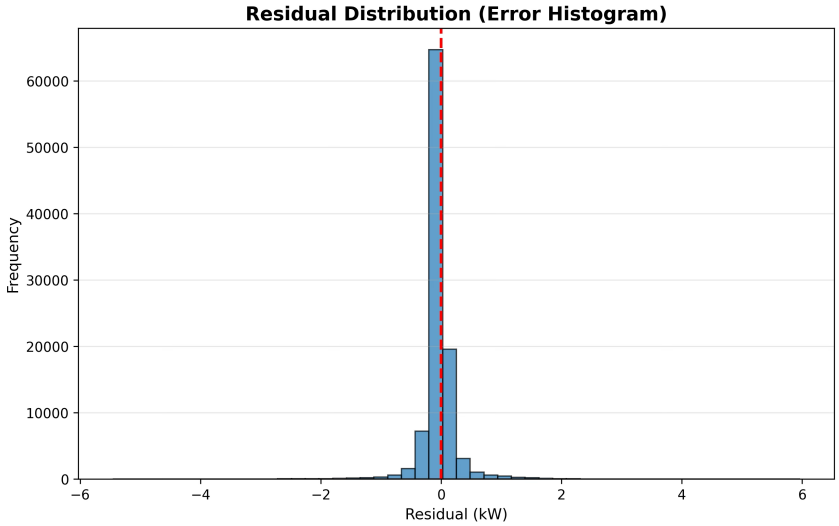


Figure 13: Histogram of Residuals demonstrating a well-calibrated error distribution with minimal bias.

7 Overview of the Application Layer

The application layer represents the interface between the end user and the underlying smart home infrastructure. While the lower layers of the system are responsible for sensing, communication, and decision making, the application layer focuses on visualization, remote control, and user interaction.

In this project, the application layer has been implemented through a mobile dashboard application, developed using the Flutter framework. The application communicates with the central Gateway through the MQTT messaging protocol, allowing real-time monitoring of environmental conditions and remote actuation of devices deployed in the smart home environment.

The main objectives of the application layer are:

- Provide real-time visualization of sensor data collected from the house.
- Allow manual remote control of actuators such as lights and heaters.
- Enable secure garage access through the BLE smart lock interface.
- Display system alerts and warnings generated by the AI-based energy monitoring system.
- Offer a unified and intuitive user interface that integrates heterogeneous communication technologies (Zigbee, LoRa, and BLE).

As illustrated in Figure 14, the system's architecture relies on distinct communication flows to manage authentication, remote telemetries, and local access control securely.

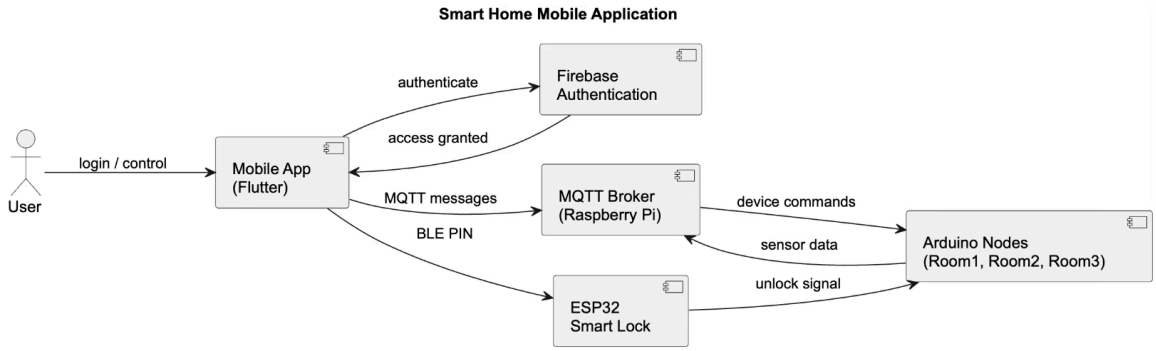


Figure 14: High-level architecture of the Smart Home Application Layer, illustrating communication flows between the Flutter mobile app, Firebase, the MQTT Broker, and the Edge nodes.

7.1 Mobile Dashboard Interface

The user interacts with the system through a mobile dashboard that displays the current status of the house and allows the user to control devices remotely. The dashboard aggregates the information collected from all nodes and displays the following parameters:

- Temperature
- Humidity

- Atmospheric pressure
- Light status
- Heater status

These values are retrieved from the Gateway via MQTT topics and updated dynamically in the application. Each room is represented as a logical block within the dashboard, providing a clear overview of the house's environmental conditions. The interface was designed following a card-based layout, where each card corresponds to a specific room or device. This design improves readability and allows users to quickly identify the operational state of the smart home.

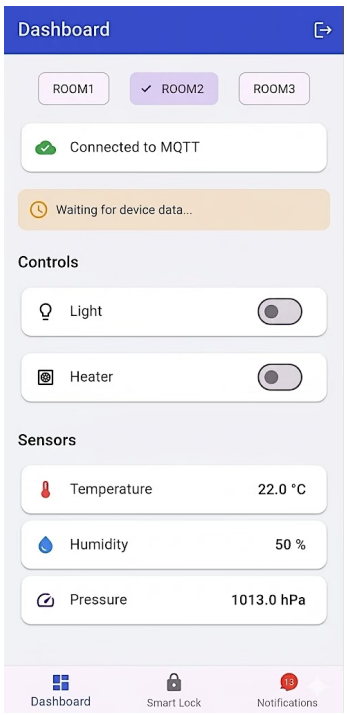


Figure 15: Mobile dashboard interface displaying real-time sensor data and actuator status.

7.2 User Authentication with Firebase

To ensure secure access to the smart home system, a user authentication mechanism was implemented using Firebase Authentication. The mobile application requires users to register and log in using their email address and password before accessing the dashboard interface. This mechanism prevents unauthorized access to the smart home control system and ensures that only authenticated users can interact with the devices.

Firebase Authentication was chosen due to its reliability, scalability, and ease of integration with Flutter applications. The authentication process is managed by Firebase's secure cloud infrastructure, which handles user credential verification and session management. Once the authentication process is completed successfully, the user is redirected to the main dashboard of the application, where real-time monitoring and device control functionalities become available. This authentication layer adds an additional level of security and access control, protecting the smart home environment from unauthorized manipulation.

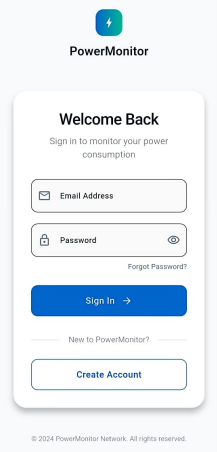


Figure 16: User Login interface.

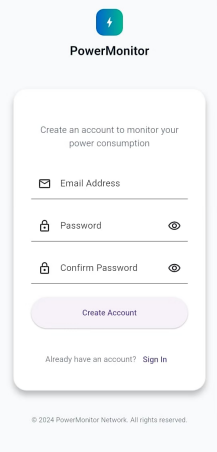


Figure 17: User Registration interface.

7.3 Real-Time Data Synchronization via MQTT

To ensure efficient and low-latency communication between the application and the Gateway, the MQTT publish/subscribe protocol was used. The Gateway publishes updates to the topic `home/status`. Each message contains a JSON payload describing the updated state of a room.

The mobile application subscribes to this topic and updates the graphical interface in real time. This asynchronous communication model ensures that the dashboard always reflects the current state of the physical environment.

7.4 Remote Device Control

In addition to monitoring the environment, the application enables the user to remotely control the actuators installed in the smart home. The control system supports:

- Light control
- Heater activation or deactivation
- Garage access control

When the user interacts with the interface (for example, pressing a button to turn on the light), the application publishes a command message to the MQTT topic `home/command`. The Gateway receives the message, identifies the correct communication protocol (Zigbee or LoRa), and forwards the translated command to the corresponding Arduino node. This architecture ensures protocol abstraction, allowing the mobile application to interact with the system without needing to know the underlying network technology.

7.5 BLE Smart Lock Interface

The application also integrates a Bluetooth Low Energy (BLE) interface for controlling the smart garage lock. When the user approaches the garage, the application scans for the BLE device. Once connected, the application allows the user to enter a PIN code. The PIN is transmitted through the BLE characteristic defined in the ESP32 firmware.

If the entered PIN matches the stored authentication code, the ESP32 activates the output signal connected to the Arduino node, which subsequently triggers the garage unlock event and sends a confirmation message to the Gateway via the LoRa network. This mechanism provides a local access control system that operates independently of the internet connection.

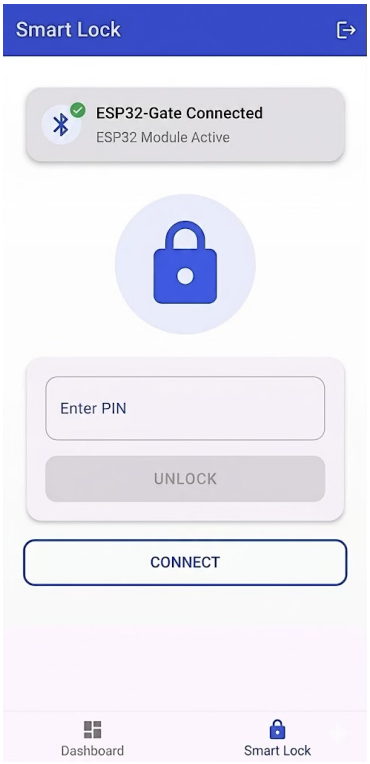


Figure 18: BLE Smart Lock interface for entering the garage access PIN.

7.6 AI-Based Alerts and Notifications

The application also acts as the visual endpoint for the AI-driven smart metering system implemented in the Gateway. When the predictive model detects abnormal energy consumption or potential overload conditions, the Gateway generates a system alert using the internal notification module.

These alerts are forwarded to the application and displayed as warning messages in the dashboard. Examples include:

- High energy consumption warnings
- Sustained overload alerts
- Emergency device shutdown notifications

This functionality transforms the application into a decision support tool, allowing the user to understand and respond to abnormal energy conditions within the smart home.

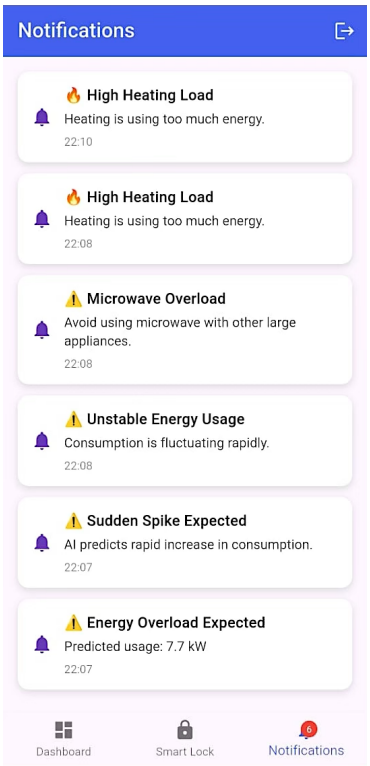


Figure 19: System alerts and AI-based notifications displayed in the application.

7.7 System Integration and User Experience

The application layer successfully integrates all components of the smart home architecture into a unified user interface. Despite the presence of heterogeneous communication technologies (Zigbee, LoRa, BLE), the user interacts with the system through a single coherent platform. This design significantly improves usability while preserving the modularity and scalability of the overall system.

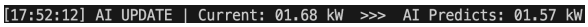
8 System Evaluation and Experimental Results

To validate the robustness, responsiveness, and proactive capabilities of the integrated Smart Home architecture, a series of experimental tests were conducted on the physical prototype.

8.1 Real-Time Energy Monitoring and AI Tracking

Before testing emergency protocols, the steady-state accuracy of the Bi-LSTM model was monitored under normal operating conditions. Figure 20 demonstrates the system's ability to accurately track fluctuations in household energy demand. Whether in a low-consumption state (approx. 0.70 kW) or during the activation of standard heating appliances (approx. 1.57 kW), the AI maintains a tight coupling between real telemetry and predicted values, providing a stable baseline for the load-shedding logic.

(a) Inference during low load.



(b) Inference during heater activation.

Figure 20: Real-time AI inference updates demonstrating the model tracking different energy states.

8.2 Edge Computing Performance and Latency

System performance was evaluated by measuring the inference latency of the Bi-LSTM model executing on the Raspberry Pi central Gateway.

As shown in Figure 21, the system achieves an average inference time of approximately 75 ms per prediction cycle. This ultra-low latency ensures that the predictive energy modeling operates seamlessly in the background.

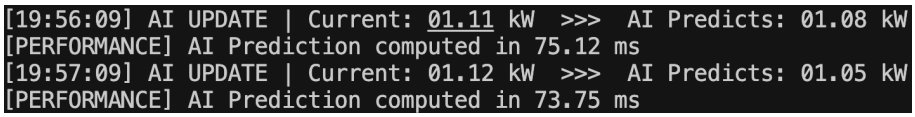


Figure 21: Terminal log demonstrating the Bi-LSTM inference execution time ($\approx$ 75 ms) on the Edge Gateway.

8.3 QoS Reliability over the LoRa Network

Figure 22 illustrates the Quality of Service (QoS) mechanism in action. When the initial transmission attempt failed resulting in a timeout, the Gateway's built-in retry logic effectively maintained the system's stability. After a 500 ms delay, the subsequent attempt successfully captured the ACK, proving the resilience of the application-layer protocol.

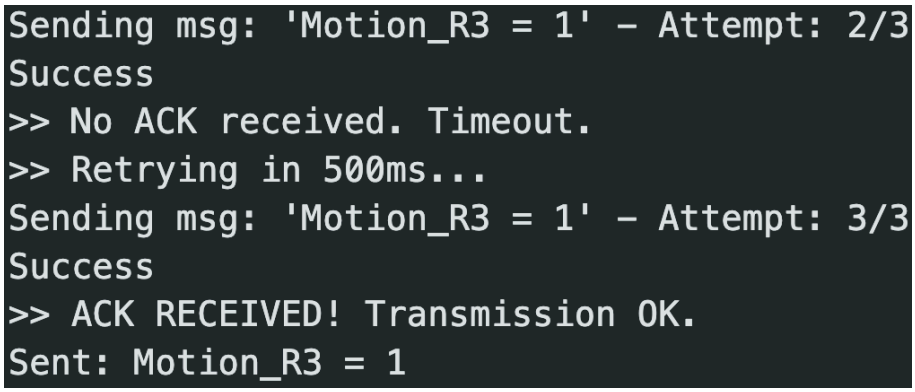


Figure 22: Terminal log demonstrating the LoRa QoS mechanism.

8.4 Predictive Load Shedding in Action

Figure 23 captures the exact moment the automated safety protocol is triggered. The Bi-LSTM model successfully predicted a sustained overload condition peaking at 3.68 kW, well above the designated 2.5 kW safety threshold. Instantly, the Gateway logged a critical alert and automatically published an MQTT command to deactivate the heater.

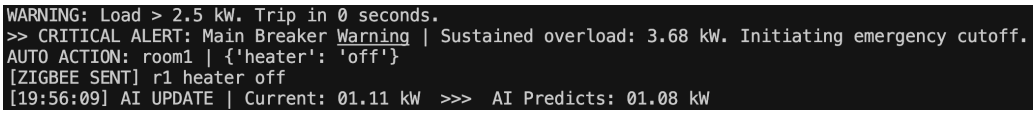


Figure 23: System log capturing the predictive load shedding event: The AI detects an imminent overload (3.68 kW) and initiates an emergency cutoff via Zigbee.

8.5 Backend Synchronization and MQTT Telemetry

Finally, the central MQTT broker's telemetry was monitored to verify the architectural decoupling. Figure 24 displays the standardized JSON payloads generated by the Gateway's state machine, providing a clean, real-time API for external clients.

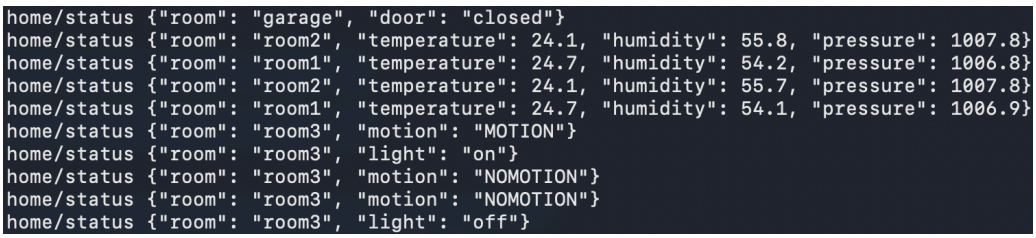


Figure 24: Real-time MQTT telemetry.

9 Conclusion

This project successfully demonstrated the end-to-end development of a highly integrated Smart Home infrastructure, seamlessly merging heterogeneous IoT networking with predictive energy modeling. By adopting a modular, multi-layer architecture, the system overcame the inherent challenges of residential automation, from physical sensor deployment to complex AI-driven decision making.

At the networking layer, the strategic deployment of Zigbee for indoor nodes and LoRa for the outdoor garage environment ensured robust, low-latency communication. The implementation of custom Quality of Service (QoS) mechanisms at the edge layer guaranteed reliable data delivery, while the central Gateway effectively acted as the system's "Single Source of Truth", bridging diverse physical protocols into a unified MQTT data stream.

From an analytical perspective, the integration of the Bidirectional LSTM network proved highly effective in capturing the non-linear, temporal dynamics of household energy consumption. Achieving a Coefficient of Determination (R^2) of 0.7317, the model successfully mapped cyclical human behaviors and environmental dependencies. More importantly, embedding this predictive engine within the Gateway transformed the system into an active Smart Meter. The successful simulation of predictive load-shedding during overload scenarios highlighted the practical viability of using AI not just for monitoring, but for automated safety and energy optimization.

Finally, the application layer successfully abstracted the system's underlying complexity. The Flutter-based mobile dashboard, fortified by Firebase authentication and BLE smart lock integration, provided a real-time, and intuitive interface for the end-user.

In conclusion, this work validates the synergy between Edge Computing, Deep Learning, and multi-protocol IoT architectures.

References

[1] Senba Sensing Technology. *AM312 PIR Motion Sensor Datasheet*. https://www.adrirobot.it/wp-content/uploads/2022/04/Datasheet-PIR-AM_312.pdf

[2] Adrirobot. *Sensore infrarosso PIR compatto AM312 - Guida all'uso*. https://www.adrirobot.it/sensore-infrarosso-pir-compatto-am312/

[3] Texas Advanced Optoelectronic Solutions. *TSL2561 Light-to-Digital Converter Datasheet*. https://cdn-shop.adafruit.com/datasheets/TSL2561.pdf

[4] Adafruit. *TSL2561 Luminosity Sensor - Arduino Code*. https://learn.adafruit.com/tsl2561/arduino-code

[5] Bosch Sensortec. *BME280 Humidity and Pressure Sensor Datasheet*. https://www.bosch-sensortec.com/media/boschsensortec/downloads/datasheets/bst-bme280-ds002.pdf

[6] Adafruit. *Adafruit BME280 Breakout - Arduino Test*. https://learn.adafruit.com/adafruit-bme280-humidity-barometric-pressure-temperature-sensor-breakout/arduino-test

[7] Chengdu Ebyte Electronic Technology Co., Ltd. *E220-900T22D Wireless Module Datasheet*. https://www.cdebyte.com/pdf-down.aspx?id=1136

[8] Xreef. *EByte LoRa E220 Series Library for Arduino*. GitHub Repository. https://github.com/xreef/EByte_LoRa_E220_Series_Library

[9] Mischianti, R. *Ebyte LoRa E220 device for Arduino, esp32 or esp8266*. https://mischianti.org/it/category/elettronica/rf-it/ebyte-lora-e220-it/

[10] Taranvee (Kaggle). *Smart Home Dataset with Weather Information*. https://www.kaggle.com/datasets/taranvee/smart-home-dataset-with-weather-information/data